AI Forward Deployed Engineer

The AI Architect & FDE Course



3 weeks to Forward Deployed Engineer

Theory Practical Projects

Week 3: Become an AI FDE

- AI Solutions Architect
- Security, Guardrails, API Gateways
- Enterprise Context Layer & Integrations
- Production Deployment
- Capstone

Week 2: Become an AI Architect

- Architecture, Evals, Observability
- Agentic RAG, MCP, Memory
- Durable State & Human in the Loop
- Agent Teams, Swarms, Fleet
- Self-Improvement, RSI

Week 1: Become an AI Engineer

- Path to FDE
- Frontier LLMs & Agents
- Coding Agents & ADLC
- Open-Source, Local, Token Cost
- Harness Engineering

Taking stock

You can already:

- ✓ Describe the role of the AI Forward Deployed Engineer
- ✓ Outline the skills and experience needed to be an AI Architect / FDE
- ✓ Build an agent from first principles with a model, a tool and a loop
- ✓ Call LLMs through OpenRouter in a lab in VS Code

By the end of today, you'll be able to:

- ○ Explain the techniques that turn LLMs into AI products
- ○ Compare models on intelligence, speed and cost
- ○ Define an AI agent and an agent harness
- ○ Build an agent using a framework like LangChain

If you're relatively new to LLMs and agents, today is a crash course.
If not, speed me up to 2x!

Recap

The roadmap to AI Architect & Forward Deployed Engineer



AI Engineering



AI Architecture



Forward Deployed Engineering



Solved real-world business problems



Delivered to customers



Instant gratification

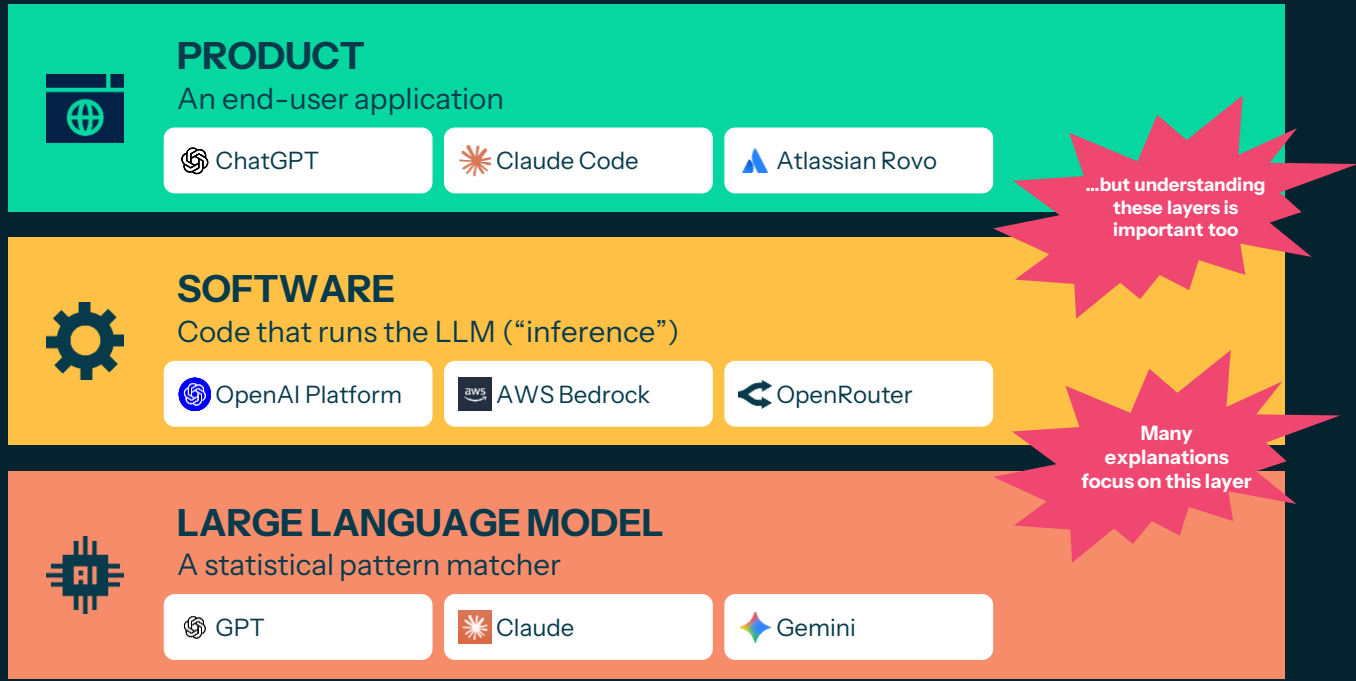
A model, a tool and a loop solved the maze



Ready to build

The repo, the packages and your key in .env

Setting the scene: What an LLM is



6 tricks that turn GPT into ChatGPT



1. Autoregression

The LLM predicts the next token.

That token is added to the input before the next prediction.

This repeats to generate the answer.



2. Chat

Trained on conversations such as:

```
User: aaa
Assistant: bbb
User: ccc
Assistant: ddd
```

Inputs use the same structure.



3. Reasoning

Training examples include reasoning:

```
User: aaa
Assistant's thinking: bbb
Assistant: ccc
```

The model still predicts tokens.



4. Conversation history

The conversation is included with each request.
This is short-term memory.



5. Context

Background information helps the model give a more accurate answer.



6. Tools

Equip an LLM with capabilities beyond generating tokens

Tokens and the context window

You're most likely very familiar with this



Token

A chunk of text, about three quarters of a word on average



Input tokens (the context)

System prompt, tools, conversation history, user prompt, relevant context



Output tokens

Generated one token at a time by the model and then included in input

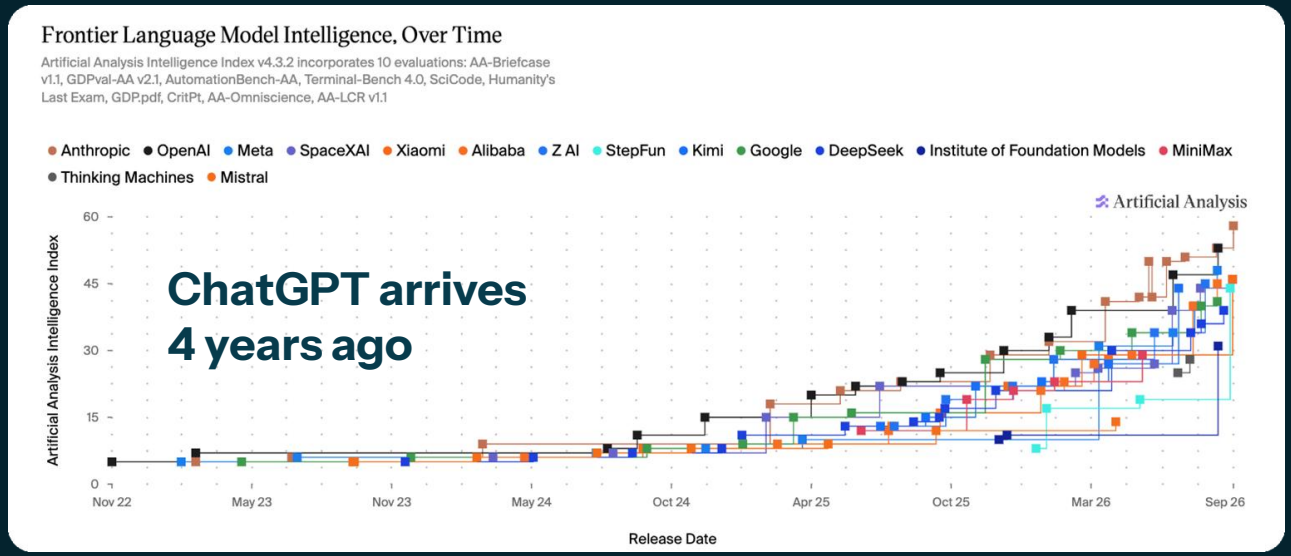


Context window

Max number of tokens a model can work with (input + output)

API prices are typically quoted per million input and output tokens
When the same inputs are sent repeatedly, you pay less ("prompt caching")
Sometimes people over-obsess over token costs. Focus on quality first..

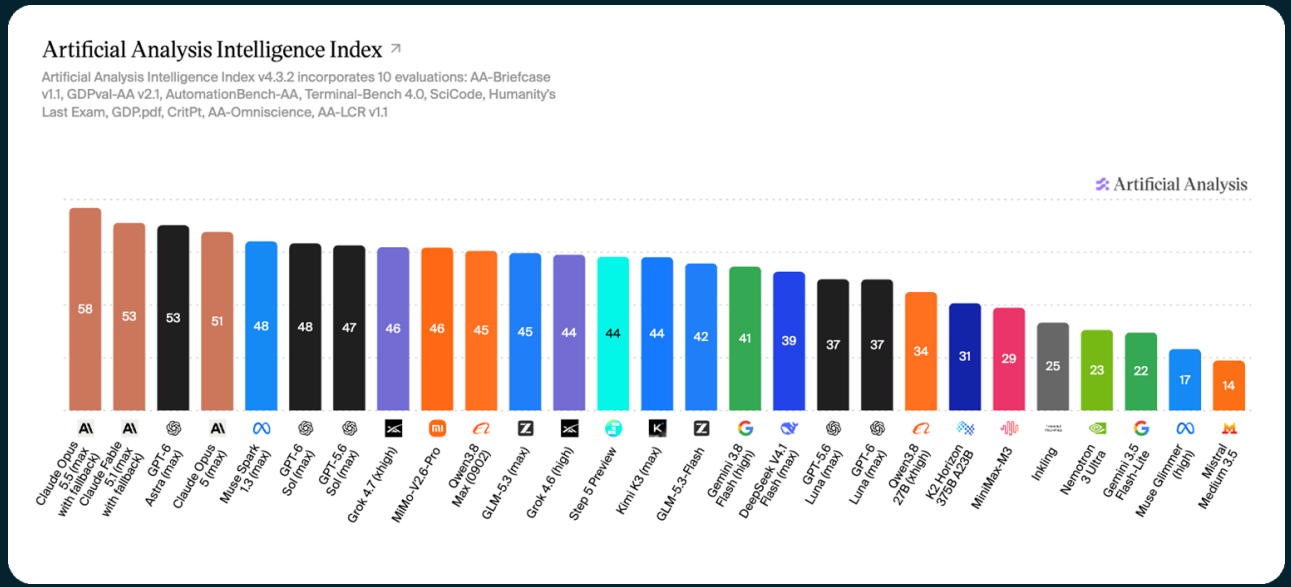
The LLM frontier over time



Pre-training | **Post-training & inference**

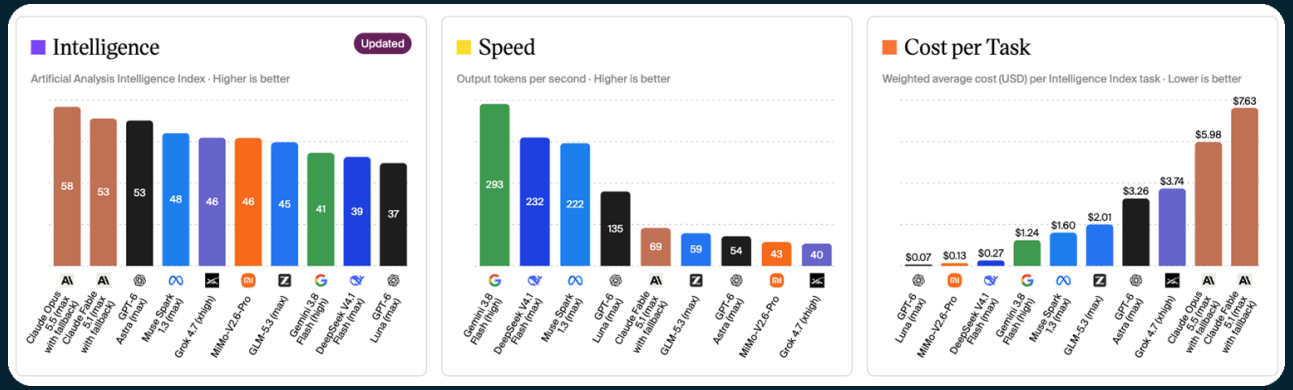
Now look at artificialanalysis.ai and see how this has changed!

The current intelligence frontier



Intelligence, speed and cost

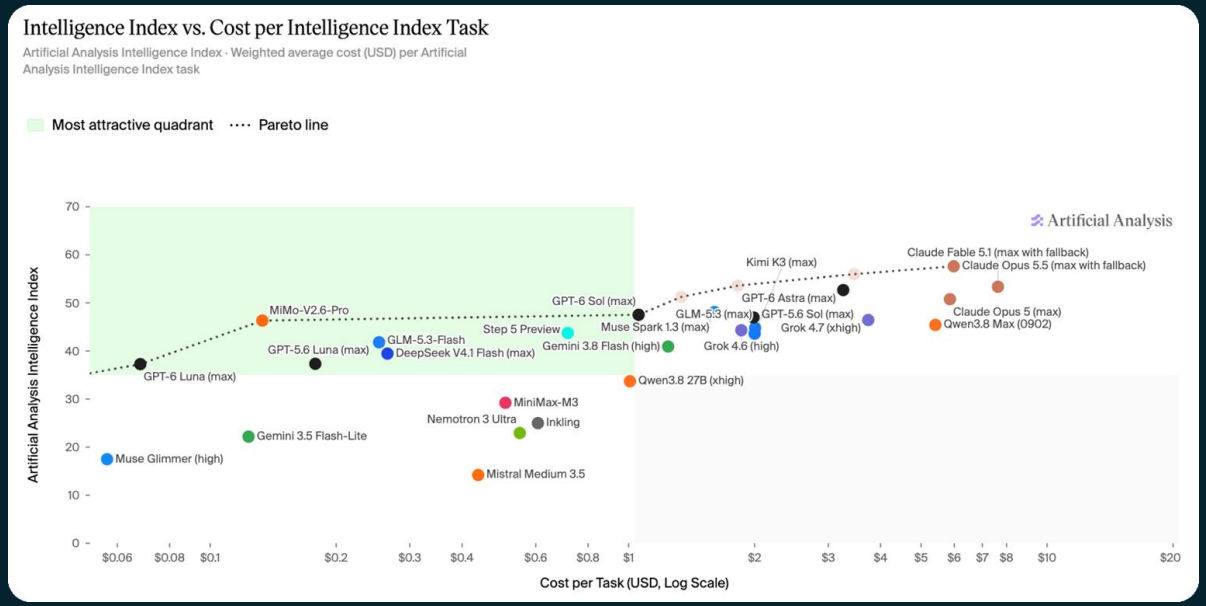
Your starting point for weighing up when selecting a model



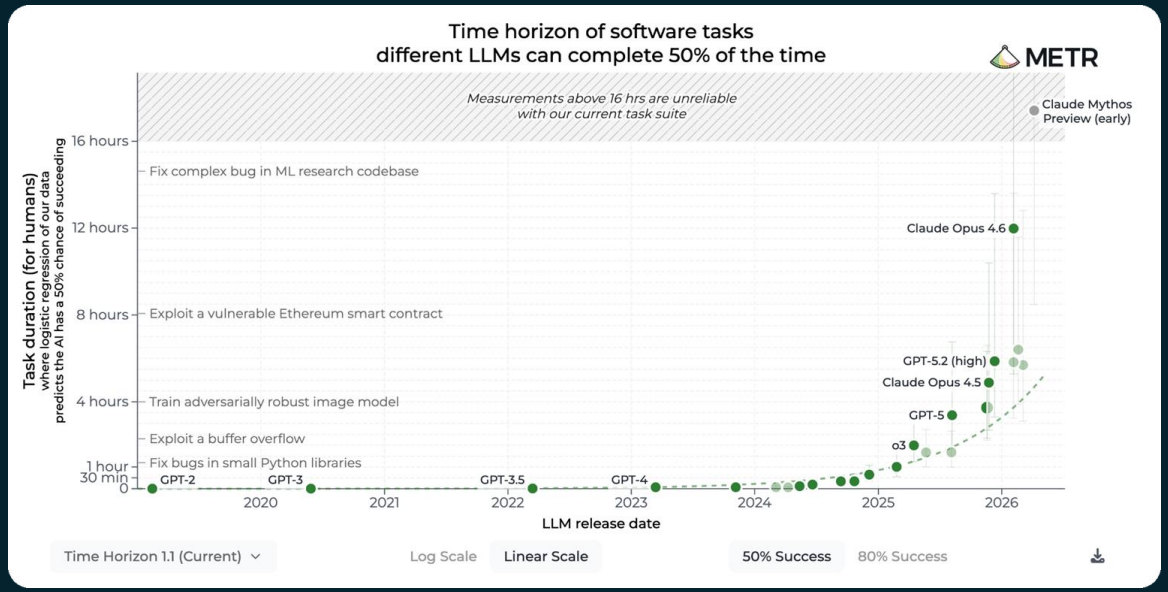
Benchmarks are useful for indicative guidance, but make real decisions based off experiments on your actual business task

In today's lab we trade off intelligence, speed and cost for a business task

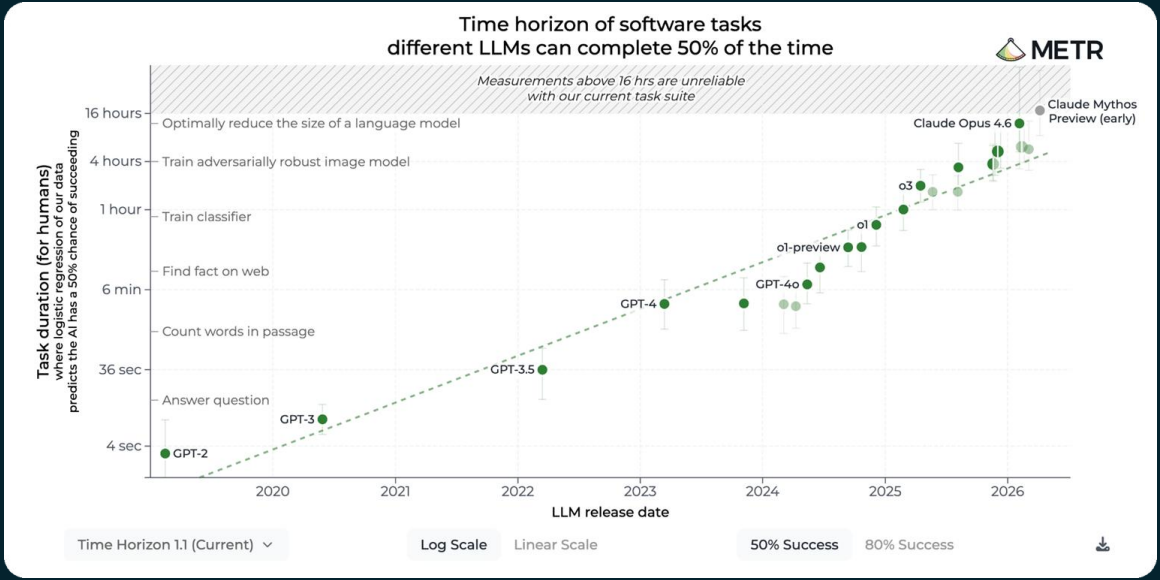
Intelligence versus cost per task



How “hard” a task can an agent do?



Exponential growth?



Three definitions of Agents



Sam Altman

“AI systems that can do work for you independently”



Early 2025 consensus (Anthropic, Hugging Face, ..)

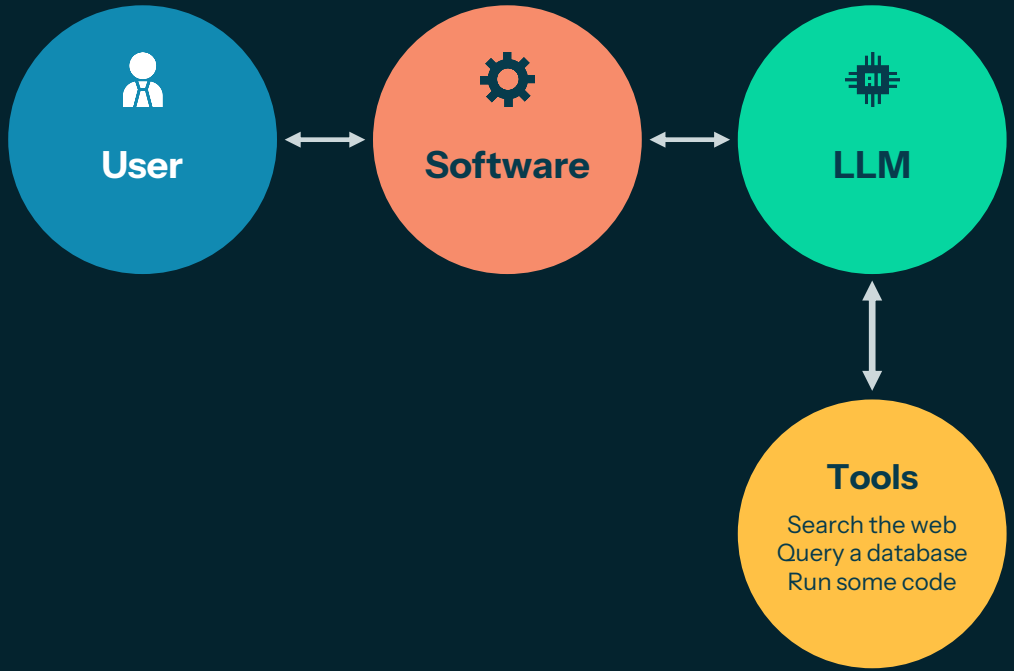
AI systems where an LLM controls the workflow



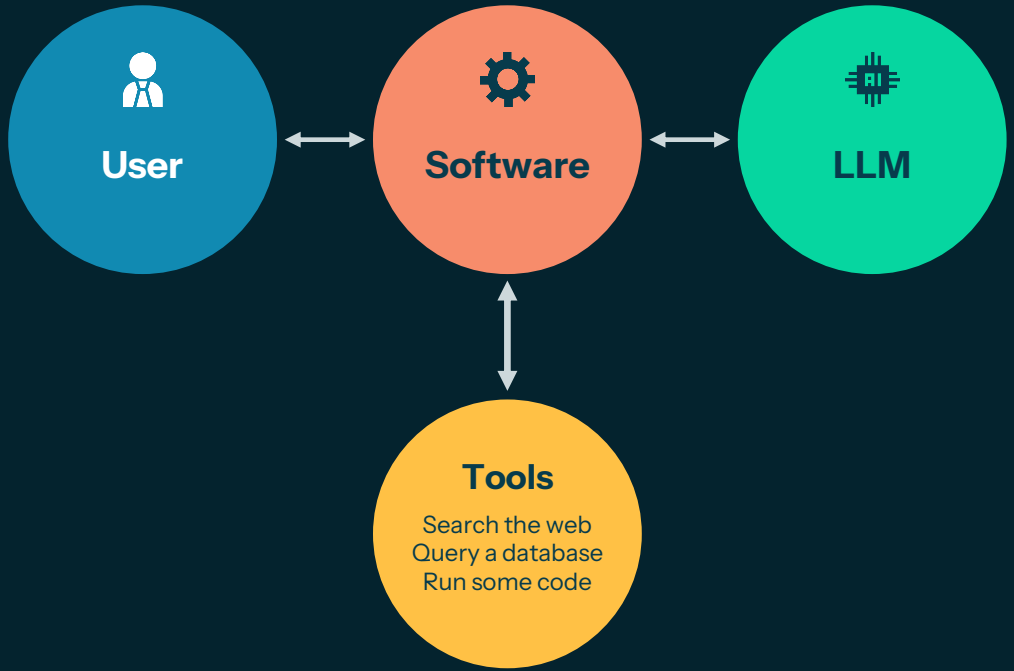
Late 2025

An LLM with Tools in a Loop to achieve a goal

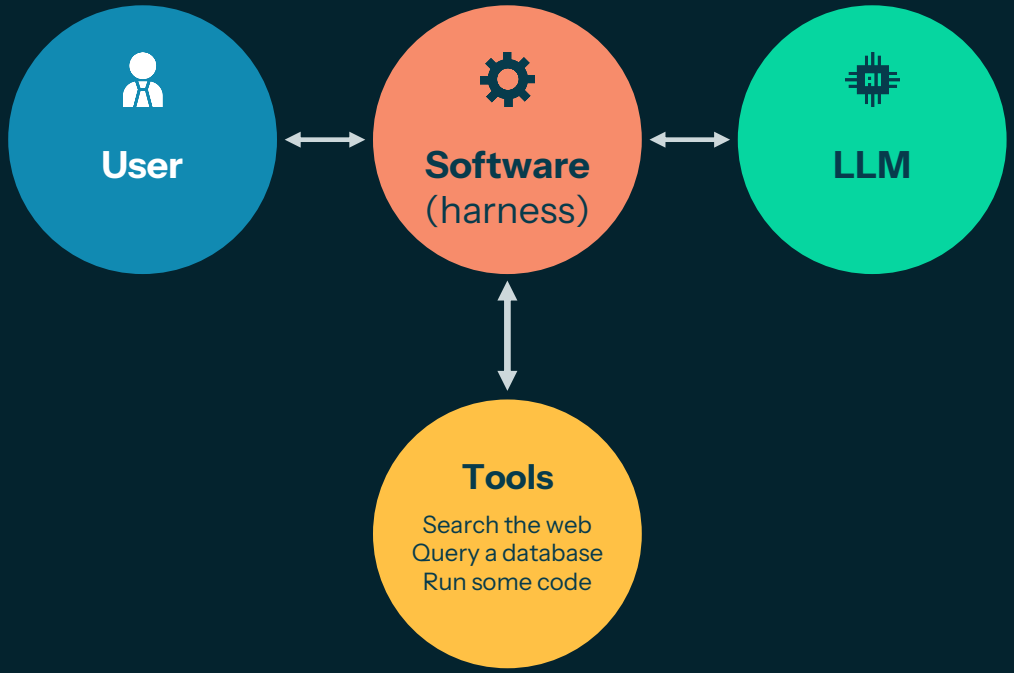
An LLM with a tool seems magical



The reality is more mundane

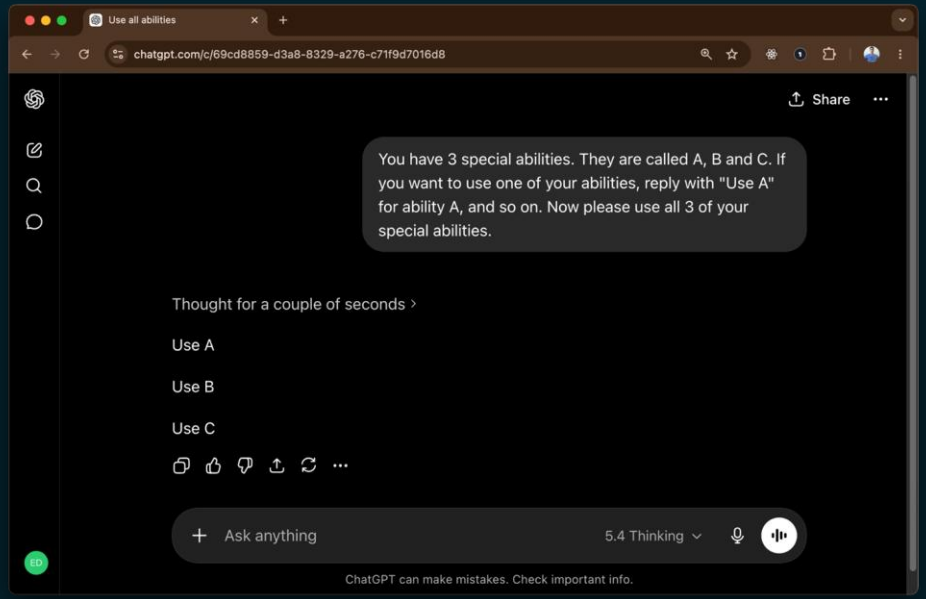


The reality is more mundane

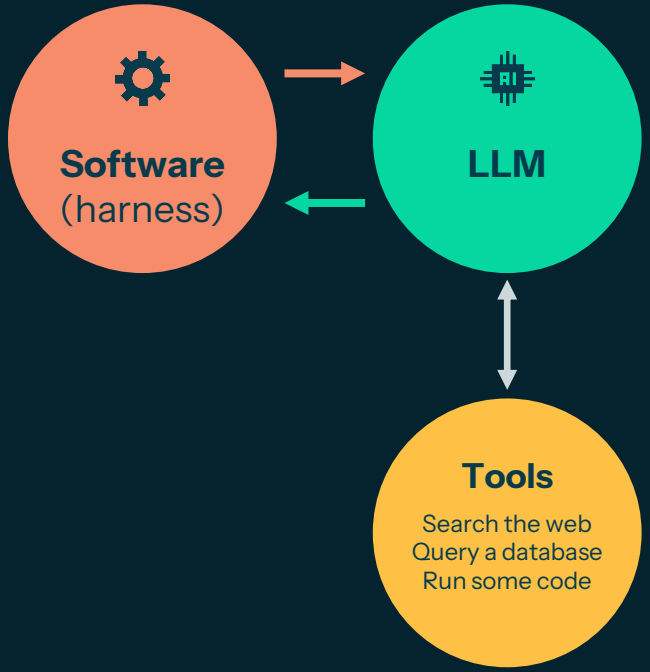


Tool use is largely clever prompting

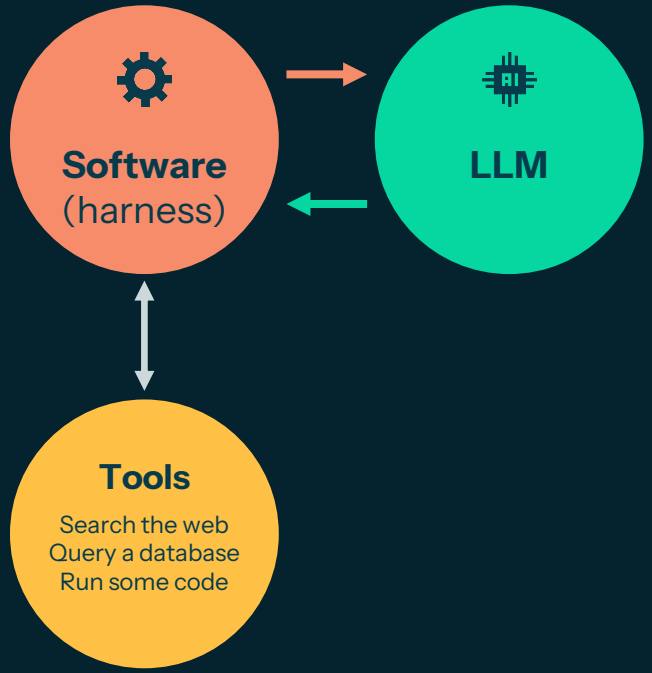
You can run this prompt yourself



The Agent Loop: it feels like this



The Agent Loop: it’s actually this



An Agent and its Harness

The definition we use throughout the course



Or: an Agent = an LLM + a Harness

The harness is everything around the model
that enables it to accomplish a more complex goal.

Which agent framework?

Not the most important decision for your project

1. **Building agents with direct LLM calls involves a lot of boilerplate code**
2. **Agent frameworks are abstraction layers that take care of common tasks**
3. **There are many agent frameworks but they largely do the same things**


Tool calling


Agent loop


Structured outputs


MCP

4. **They sit on a spectrum from lightweight to opinionated & batteries-included**


Built-in tools

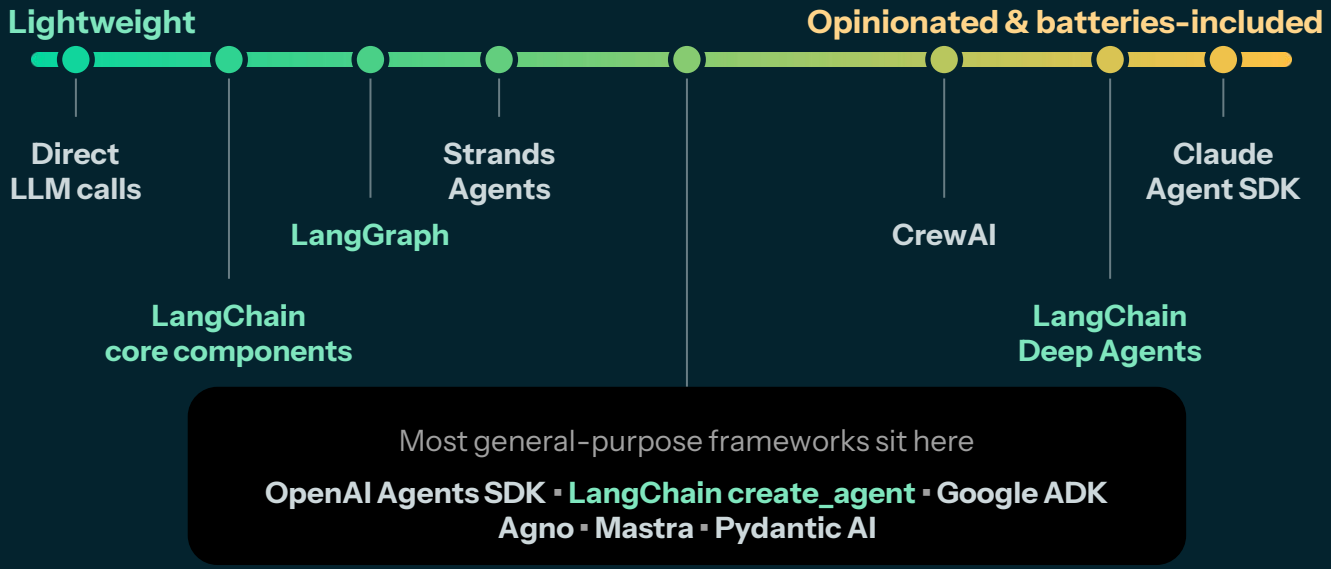

Guardrails


Observability


Sandboxes


Human in the loop

The spectrum of agent frameworks



LangChain and LangGraph were the most common in the job postings I examined

Introducing UnderCover

Our client for the next three weeks is a fictional travel insurer



The pilot brief

The brief from the UnderCover's claims team



A queue of claims decided by hand

Handlers read each claim, check the policy, then approve, decline or refer



The pilot: an agent that decides claims

It works the queue like a handler and explains each decision



Success: claims handled straight through

As accurate as the handlers, with a human approving payouts over $1,000

Today we put the first claims to LLMs in a notebook

The UnderCover roadmap

The most ambitious, enterprise-strength project in any of my courses



Week 1: Become an AI Engineer

LLMs decide claims in a notebook, then Claude Code builds the backend, the dashboard and the agent; open-source models; harness engineering



Week 2: Become an AI Architect

Evals and observability, agentic RAG and memory, a human approval step, agent teams and an agent that improves itself



Week 3: Become an AI FDE

Security and guardrails, enterprise context and integrations, then deployment on AWS AgentCore and a handover to the client

Build it as is, improve it or take it in your own direction

Today's lab

In the course repo: `labs/week1day2.ipynb`

1. **Examine the data**
   25 insurance claims along with the correct decisions

2. **Run with some models**
   Via OpenRouter with structured outputs

3. **Optional: larger models and Jev**
   GPT-6 Sol at high reasoning and TypeSafe's Jev

4. **An Agent via LangChain**
   Looks up the policy and records its decision with tools

Common LLM API features we will use

1

Chat Completions API

`messages` is a Python list of dicts, each with a role and content

```
messages = [{"role": "system", "content": system_prompt},
            {"role": "user", "content": user_prompt}]
openai.chat.completions.create(model=..., messages=messages)
```

2

Structured outputs

Describe the JSON schema as a Pydantic class and pass it to `parse()`

```
class ClaimDecision(BaseModel):
    rationale: str
    decision: Literal["approve", "decline", "refer"]
    payout: float

openai.chat.completions.parse(..., response_format=ClaimDecision)
```

Preparing for the first main lab

How the labs work

1. **The notebooks are filled in already**
   I step through them, explaining and changing the code as I run it

2. **Make them your own**
   Change things as you go, add print statements and try your own experiments

3. **Some code isn't about AI**
   I may run the charts and other analysis cells without explaining every line

4. **Before we start: add a little OpenRouter credit**
   If you'd rather not spend anything, use `openrouter/free`

*Alternative lab **week1day2free.ipynb** with free models throughout*

LangChain `create_agent`

A lightweight agent framework within LangChain

1

Describe the LLM with a LangChain class like `ChatOpenAI`

```
llm = ChatOpenAI(model="openai/gpt-6-luna", base_url=...)
```

2

Turn a function into a tool with `@tool`

```
@tool
def get_policy(policy_id: str):
```

3

`create_agent` builds the agent and `invoke()` runs its loop

```
agent = create_agent(model=llm, tools=[get_policy, record_decision])
agent.invoke({"messages": [{"role": "user", "content": prompt}]})
```

Behind the scenes, a LangGraph graph is created; we'll find out more in Week 2

3 weeks to Forward Deployed Engineer

Theory Practical Projects

Week 3: Become an AI FDE

AI Solutions Architect	Security, Guardrails, API Gateways	Enterprise Context Layer & Integrations	Production Deployment	Capstone

Week 2: Become an AI Architect

Architecture, Evals, Observability	Agentic RAG, MCP, Memory	Durable State & Human in the Loop	Agent Teams, Swarms, Fleet	Self-Improvement, RSI

Week 1: Become an AI Engineer

Path to FDE	Frontier LLMs & Agents	Coding Agents & ADLC	Open-Source, Local, Token Cost	Harness Engineering

AI Forward Deployed Engineer

The AI Architect & FDE Course



13% Complete